

Tóm tắt tài liệu "Agent2Agent Protocol" (A2A) một cách logic để dễ hiểu trong 1 giờ.

I. Tổng quan về A2A Protocol

1. Vấn đề: [cite_start]Sự hợp tác giữa các Agent độc lập, được xây dựng trên các nền tảng công nghệ khác nhau (như LangGraph, CrewAI, Llamaindex), là một thách thức lớn trong môi trường doanh nghiệp hiện đại[cite: 10, 12, 13, 14, 15, 16].

- [cite_start]Không có một tiêu chuẩn giao tiếp chung, dẫn đến việc tích hợp các Agent trở nên phức tạp, thiếu ổn định, và đòi hỏi phải tạo các lớp chuyển đổi riêng cho từng cặp Agent[cite: 20, 21].
- [cite_start]Bất kỳ thay đổi nào về thành phần Agent hoặc cập nhật API đều có thể gây lỗi dây chuyền, ảnh hưởng đến tính ổn định của quy trình hợp tác liên Agent[cite: 22].
- [cite_start]Hệ thống thiếu tính linh hoạt, dễ bị phân tán chức năng, cản trở việc mở rộng quy mô hoặc thích ứng với yêu cầu nghiệp vụ mới[cite: 23].

2. Giải pháp: Agent-to-Agent (A2A) Protocol [cite_start]A2A Protocol là một giao thức tiêu chuẩn, được thiết kế để giải quyết toàn diện các thách thức trong môi trường AI đa Agent[cite: 30, 38].

- [cite_start]A2A định nghĩa một tập hợp quy tắc giao tiếp và định dạng thông điệp chuẩn hóa, đóng vai trò như một lớp trừu tượng (abstraction layer) giữa các Agent[cite: 39, 43].
- [cite_start]Giúp che giấu chi tiết về ngôn ngữ lập trình, framework, hoặc cách triển khai nội bộ của từng Agent cụ thể[cite: 43].
- [cite_start]Nhờ đó, các Agent có thể liên kết, phối hợp và hoán đổi linh hoạt, bất kể sự khác biệt về công nghệ nền tảng, tăng cường khả năng tương tác liên nền tảng và thúc đẩy khả năng mở rộng[cite: 44, 45].

3. Các yếu tố cần thiết cho sự hợp tác hiệu quả giữa các Agent:

- [cite_start]**Khám phá năng lực (Capability Discovery):** Mỗi Agent phải công bố rõ các tác vụ mình hỗ trợ, định dạng dữ liệu chấp nhận và phương thức giao tiếp[cite: 25].
- [cite_start]**Thỏa thuận cách tương tác (UX Negotiation):** Các Agent cần thống nhất về hình thức trao đổi thông tin (văn bản, biểu mẫu, file, stream...)[cite: 26].
- [cite_start]**Quản lý tác vụ và trạng thái (Task and State Management):** Mỗi tác vụ phải được theo dõi rõ ràng trong suốt vòng đời (submitted → working → completed), kể cả khi qua nhiều Agent xử lý[cite: 27].
- [cite_start]**Hợp tác an toàn (Secure Collaboration):** Giao tiếp giữa các Agent phải đảm bảo xác thực, phân quyền và bảo mật[cite: 28].

II. Các khái niệm cốt lõi của A2A Protocol

1. Agent Card

- [cite_start]Là một tệp JSON công khai đóng vai trò như "danh thiếp kỹ thuật số" của một Agent tuân thủ giao thức A2A[cite: 76].
- [cite_start]Chứa thông tin mô tả về Agent và thường được lưu trữ tại đường dẫn `/.well-known/agent.json` [cite: 76].
- [cite_start]**Mục đích:** Phục vụ cho việc khám phá Agent (Agent Discovery), giúp các client truy xuất tệp này để tìm hiểu năng lực và phương thức tương tác của Agent[cite: 77, 78].

- [cite_start]**Các trường chính:** `name`, `description`, `url`, `version`, `capabilities` (streaming, pushNotifications, stateTransitionHistory), `authentication`, `defaultInputModes/defaultOutputModes`, `skills` (id, name, description, inputModes/outputModes, examples)[cite: 85, 87, 89, 90, 92, 93, 97, 98, 99, 100, 101, 102, 103, 105, 107, 108, 109, 110, 111].

2. Giao thức JSON-RPC 2.0

- [cite_start]A2A sử dụng JSON-RPC 2.0 làm nền tảng chuẩn để các Agent giao tiếp thống nhất, không phụ thuộc nền tảng[cite: 161].
- [cite_start]Cho phép client gửi yêu cầu, theo dõi tiến trình, nhận phản hồi hoặc hủy bỏ các tác vụ (task)[cite: 162].
- **Các phương thức chính:**
 - [cite_start]`tasks/send`: Gửi task mới và nhận phản hồi khi hoàn tất[cite: 164].
 - [cite_start]`tasks/sendSubscribe`: Gửi task và nhận tiến trình xử lý theo thời gian thực qua SSE[cite: 165].
 - [cite_start]`tasks/get`: Truy vấn trạng thái hiện tại của một task[cite: 166].
 - [cite_start]`tasks/cancel`: Hủy task đang xử lý[cite: 167].
 - [cite_start]`tasks/pushNotification/set`: Cấu hình webhook để nhận thông báo chủ động[cite: 168].
 - [cite_start]`tasks/pushNotification/get`: Xem lại webhook đã đăng ký[cite: 169].
 - [cite_start]`tasks/resubscribe`: Khôi phục luồng SSE nếu bị ngắt kết nối[cite: 170].

3. A2A Server

- [cite_start]Thành phần được triển khai kèm theo mỗi Agent chuyên biệt (LangGraph, CrewAI, LlamaIndex, v.v.)[cite: 195].
- [cite_start]Đảm nhiệm vai trò tiếp nhận và xử lý các tác vụ từ A2A Client thông qua JSON-RPC[cite: 195].
- **Trách nhiệm chính:**
 - [cite_start]Xử lý các phương thức JSON-RPC (`tasks/send`, `tasks/get`, `tasks/cancel`) [cite: 216, 217, 218, 219].
 - [cite_start]Quản lý vòng đời tác vụ (submitted → working → completed / failed)[cite: 220, 221].
 - [cite_start]Gửi phản hồi đến client qua HTTP response, SSE, hoặc Webhook[cite: 222, 223, 224, 225].
 - [cite_start]Liên kết trực tiếp với logic xử lý nội bộ của Agent (phân tích log, tổng hợp dữ liệu, triển khai bản vá)[cite: 228, 229, 230, 231].

4. A2A Client

- [cite_start]Thành phần trung gian điều phối tác vụ đến các A2A Server đã đăng ký thông qua JSON-RPC 2.0[cite: 237].
- **Thực hiện:**
 - [cite_start]Gửi các lệnh JSON-RPC (`tasks/send`, `tasks/sendSubscribe`)[cite: 239].
 - [cite_start]Lắng nghe phản hồi từ Server qua HTTP response, SSE, hoặc Webhook[cite: 240, 241, 242, 243].
 - [cite_start]Tái kết nối hoặc đăng ký lại stream khi bị gián đoạn[cite: 244].

- [cite_start]Không cần biết công nghệ nội bộ của Agent, chỉ cần đọc Agent Card và điều phối qua A2A[cite: 245].

5. Task

- [cite_start]Là một yêu cầu công việc do client khởi tạo, được agent xử lý[cite: 261]. [cite_start]Giao tiếp trong A2A luôn xoay quanh Task[cite: 262].
- [cite_start]**Vòng đời Task (Task Lifecycle):** `submitted`, `working`, `input-required`, `completed`, `failed`, `canceled`, `unknown`[cite: 263, 264, 265, 266, 267, 268, 269, 270].
- [cite_start]**Thông tin chính:** `id`, `sessionId`, `status`, `artifacts`, `history`, `metadata`[cite: 271, 272, 273, 274, 275, 276, 277].

6. Message

- [cite_start]Đại diện cho một lượt hội thoại trong quá trình xử lý một Task[cite: 361].
- [cite_start]**Cấu trúc:** `role` ("user" hoặc "agent"), `parts` (danh sách các Part), `metadata` (tùy chọn)[cite: 363, 364, 365].

7. Part

- [cite_start]Là đơn vị nội dung cơ bản trong một Message hoặc Artifact[cite: 370]. [cite_start]Một Message có thể chứa nhiều Part với các kiểu khác nhau[cite: 371].
- [cite_start]**Các loại Part:** `TextPart` (văn bản thuần), `FilePart` (tệp tin, có thể gồm bytes hoặc uri), `DataPart` (dữ liệu JSON có cấu trúc)[cite: 372, 373, 374, 375, 376, 377]. [cite_start]Mỗi Part có thể đính kèm `metadata` (tùy chọn)[cite: 378].

8. Artifact

- [cite_start]Đại diện cho kết quả do Agent tạo ra trong quá trình thực hiện task, khác biệt với hội thoại (Message)[cite: 380]. [cite_start]Ví dụ: mã nguồn, ảnh, tài liệu, dữ liệu có cấu trúc[cite: 381].
- [cite_start]**Cấu trúc:** `name/description` (tùy chọn), `parts` (bắt buộc), `metadata` (tùy chọn), `index`, `append`, `lastChunk` (tùy chọn, dùng khi streaming)[cite: 382, 383, 384, 385, 386].

III. Mô hình tương tác và phối hợp Agent

1. Mô hình tương tác giữa 2 Agent (A2A Workflow) [cite_start]Một phiên giao tiếp tiêu chuẩn giữa hai Agent trong A2A thường trải qua 5 giai đoạn chính[cite: 409]:

- **1. Khám phá năng lực (Discovery):**
 - [cite_start]Mục tiêu: Client Agent hiểu được Remote Agent có thể làm gì[cite: 410, 411].
 - [cite_start]Cách thức: Client Agent thực hiện HTTP GET tới endpoint `/.well-known/agent.json`[cite: 412].
 - [cite_start]Kết quả: Nhận về Agent Card chứa thông tin định danh, năng lực tác vụ, endpoint API, định dạng dữ liệu hỗ trợ và cơ chế xác thực[cite: 413].
- **2. Giao nhiệm vụ (Task Assignment):**
 - [cite_start]Mục tiêu: Khởi tạo một task mới để Remote Agent xử lý[cite: 414, 415].
 - [cite_start]Cách thức: Client gửi yêu cầu thông qua phương thức `tasks/send` hoặc `tasks/sendSubscribe`[cite: 416].
 - [cite_start]Nội dung: Task bao gồm tên tác vụ, dữ liệu đầu vào, mô tả yêu cầu và các cấu hình liên quan[cite: 417].

- [cite_start]Chuẩn giao tiếp: Tuân theo JSON-RPC 2.0[cite: 418].
- **3. Trao đổi dữ liệu (Communication):**
 - [cite_start]Mục tiêu: Hỗ trợ truyền tải dữ liệu bổ sung hoặc tương tác hai chiều trong quá trình xử lý task[cite: 419, 420].
 - [cite_start]Cách thức: Gửi các phần dữ liệu nhỏ (Message Part) qua lại giữa hai Agent[cite: 424].
 - [cite_start]Dữ liệu hỗ trợ: văn bản, biểu mẫu có cấu trúc, file đính kèm, URI tham chiếu[cite: 425].
- **4. Cập nhật tiến trình (Task Progress):**
 - [cite_start]Mục tiêu: Client theo dõi trạng thái của task theo thời gian thực[cite: 426, 427].
 - [cite_start]Phương pháp: Remote Agent gửi các cập nhật định kỳ hoặc theo sự kiện về trạng thái tác vụ (**submitted**, **working**, **completed**, **failed**)[cite: 428].
 - [cite_start]Cơ chế truyền: Thông qua SSE (Server-Sent Events) hoặc webhook[cite: 429].
- **5. Hoàn tất và trả kết quả (Completion):**
 - [cite_start]Mục tiêu: Cung cấp đầu ra cuối cùng cho Client sau khi task hoàn thành[cite: 430, 431].
 - [cite_start]Dữ liệu trả về: Có thể là văn bản, tệp, dữ liệu JSON hoặc URI đến artifact[cite: 432].
 - [cite_start]Cách truyền: Gửi kèm theo cập nhật trạng thái cuối, hoặc thông qua cơ chế pull[cite: 433].

2. Mô hình phối hợp nhiều Agent [cite_start]Kiến trúc hệ thống đa Agent (multi-agent system) sử dụng A2A để giao tiếp và điều phối tác vụ[cite: 468].

- **Quy trình 1: Khởi tạo và khám phá Agent chuyên biệt**
 - [cite_start]Mỗi Agent chuyên biệt (LangGraph, CrewAI, Google ADK, v.v.) hoạt động như một A2A Server và công bố năng lực thông qua Agent Card (**`/well-known/agent.json`**)[cite: 474].
 - [cite_start]Host Agent chứa các A2A Client, kết nối đến các A2A Server của các Agent chuyên biệt (ví dụ: Weather Agent, Airbnb Agent)[cite: 475].
 - [cite_start]A2A Client truy xuất Agent Card để biết khả năng, endpoint và định dạng giao tiếp[cite: 476].
 - [cite_start]Kết quả: Hệ thống sẵn sàng phối hợp với nhiều Agent không đồng nhất mà không cần viết tay từng lớp tích hợp riêng[cite: 477].
- **Quy trình 2: Từ yêu cầu người dùng đến phản hồi cuối cùng**
 - [cite_start]Người dùng nhập yêu cầu từ giao diện web, gửi về Host Agent[cite: 479].
 - [cite_start]Host Agent phân tích yêu cầu và chọn Remote Agent phù hợp, dựa trên thông tin đã khám phá từ Agent Card[cite: 480].
 - [cite_start]Remote Agent gửi task đến Agent Server qua các phương thức **`tasks/send`** hoặc **`tasks/sendSubscribe`**[cite: 481].
 - [cite_start]Trong quá trình xử lý, Agent Server cập nhật tiến trình và kết quả qua luồng SSE hoặc webhook[cite: 482].
 - [cite_start]Host Agent tổng hợp dữ liệu đầu ra và gửi phản hồi cuối cùng về frontend cho người dùng[cite: 483].

IV. Xây dựng mô hình áp dụng A2A và MCP (Multi-Agent Cooperation Protocol)

[cite_start]Mô hình này minh họa cách các Agent chuyên biệt (Airbnb Agent, Weather Agent) tương tác với Host Agent thông qua A2A, đồng thời sử dụng MCP để gọi các công cụ (toolset) thực thi nghiệp vụ[cite: 484].

510, 511].

1. Tổng quan về mô hình:

- [cite_start]Hệ thống được thiết kế theo kiến trúc đa Agent phân tán, trong đó các thành phần Agent tương tác thông qua A2A Protocol dựa trên JSON-RPC over HTTP[cite: 510].
- [cite_start]**Routing Agent (Host Agent)**: Trung tâm hệ thống, tiếp nhận truy vấn từ người dùng (qua giao diện Gradio), phân tích mục đích và định tuyến tác vụ đến các Remote Agent chuyên biệt[cite: 511, 512].
- **Remote Agents (Weather Agent, Airbnb Agent)**:
 - **Weather Agent**: Xử lý truy vấn khí tượng. Nhận truy vấn qua A2A Server, sử dụng LLM để phân tích và gọi công cụ từ MCP toolset để thực hiện API call đến weather.gov. [cite_start]Dữ liệu trả về được phân tích, tóm tắt và gửi lại Host Agent qua A2A[cite: 516, 517, 518, 519, 520, 521].
 - **Airbnb Agent**: Xử lý truy vấn chỗ ở. Nhận truy vấn từ Host Agent qua A2A, dùng LLM phân tích yêu cầu, trích xuất tham số, và gọi công cụ từ MCP toolset để truy xuất dữ liệu từ API Airbnb. [cite_start]Kết quả được phân tích và phản hồi lại Host Agent qua A2A[cite: 522, 523, 524, 525].
- [cite_start]Mỗi Remote Agent được tổ chức độc lập với A2A Server, tool xử lý, và phiên làm việc ngữ nghĩa riêng[cite: 529].
- [cite_start]A2A Protocol đóng vai trò cầu nối, đảm bảo hợp tác liên-agent mà không cần chia sẻ logic nội bộ hay tài nguyên, giúp hệ thống có tính mở rộng cao[cite: 530, 531].

2. Các bước xây dựng A2A Server (ví dụ: Airbnb Agent) Việc xây dựng một A2A Server bao gồm các thành phần chính:

- [cite_start]**Phần 1: Build Agent Airbnb ([airbnb_agent/airbnb_agent.py](#))** [cite: 533]
 - [cite_start]Minh họa quá trình tạo ra một Agent có khả năng nhận yêu cầu, sử dụng mô hình ngôn ngữ (LLM) và toolset (từ MCP) để phản hồi theo định dạng chuẩn của A2A[cite: 534].
 - [cite_start]Sử dụng thư viện [langgraph](#) để xây dựng Agent theo mô hình ReAct (tool-augmented reasoning)[cite: 551].
 - [cite_start]Định nghĩa format phản hồi ([ResponseFormat](#)) với các trạng thái [completed](#), [input_required](#), [error](#)[cite: 561, 563, 564, 565, 566, 567, 568].
 - [cite_start]Lớp [AirbnbAgent](#) chứa các chỉ dẫn hệ thống ([SYSTEM_INSTRUCTION](#), [RESPONSE_FORMAT_INSTRUCTION](#)) và các phương thức [ainvoke\(\)](#) (xử lý không stream) và [stream\(\)](#) (phản hồi theo thời gian thực)[cite: 570, 571, 595, 652].
- [cite_start]**Phần 2: Build Airbnb Agent Executor ([airbnb_agent/agent_executor.py](#))** [cite: 738]
 - [cite_start]Là thành phần trung gian giữa A2A Server và Agent[cite: 739].
 - [cite_start]Nhận yêu cầu từ A2A Server (qua [DefaultRequestHandler](#))[cite: 741].
 - [cite_start]Gọi hàm [agent.stream\(...\)](#) để xử lý câu hỏi người dùng[cite: 744].
 - [cite_start]Đẩy kết quả phản hồi về [EventQueue](#) theo định dạng A2A Event ([TaskStatusUpdateEvent](#), [TaskArtifactUpdateEvent](#))[cite: 746, 747].
 - [cite_start]Lớp [AirbnbAgentExecutor](#) kế thừa từ [AgentExecutor](#) và override hàm [execute\(\)](#) để xử lý task và [cancel\(\)](#) để xử lý hủy tác vụ[cite: 776, 788, 792, 844].
- [cite_start]**Phần 3: Build Airbnb A2A Server ([airbnb_agent/main.py](#))** [cite: 854]
 - [cite_start]Khởi tạo và chạy một server tuân theo chuẩn A2A[cite: 856].
 - [cite_start]Khởi tạo [AgentExecutor](#) có kết nối tool từ MCP[cite: 857].
 - [cite_start]Xây dựng A2A app với lifecycle đầy đủ[cite: 858].

- [cite_start]Đăng ký metadata Agent (**AgentCard**) để UI A2A hiểu và tương tác đúng[cite: 859].
- [cite_start]Sử dụng **A2AStarletteApplication** để tạo ứng dụng ASGI tương thích với A2A Platform và **uvicorn** để chạy server[cite: 870, 967, 977, 988].
- [cite_start]**Phần 4: Build Weather A2A server (**weather_agent**)** [cite: 992]
 - [cite_start]Tương tự Airbnb A2A Server, một A2A server thứ hai được khởi tạo cho **weather_agent**[cite: 1002].
 - [cite_start]Agent này dựa trên LLM và Google ADK SDK, tương tác với người dùng để truy vấn thông tin thời tiết[cite: 1003].
 - [cite_start]Sử dụng **LlmAgent**, mô hình **gemini-2.5-flash**, và **MCPToolset** để giao tiếp với **weather_mcp.py** qua **stdio**[cite: 1006, 1007, 1008].
 - [cite_start]Hỗ trợ streaming nội dung, push notification và tương thích hoàn toàn với giao diện người dùng A2A[cite: 1016].

3. Xây dựng Host Agent Host Agent là trung tâm điều phối, định tuyến yêu cầu người dùng đến các Remote Agent.

- [cite_start]**Phần 1: Build Remote Agent Connection (**host_agent/remote_agent_connection.py**)** [cite: 1018]
 - [cite_start]Thiết lập kết nối từ xa đến các Agent con trong kiến trúc A2A[cite: 1019].
 - [cite_start]Là trung gian giữa Host Agent và các Agent khác (**weather_agent**, **airbnb_agent**) [cite: 1020].
 - [cite_start]Sử dụng **httpx.AsyncClient** để gửi yêu cầu và **A2AClient** để chuẩn hóa giao tiếp với Agent qua **AgentCard**[cite: 1041, 1042].
 - [cite_start]Quản lý **A2AClient**, lưu thông tin từ **AgentCard**, và giao tiếp với Agent con qua **send_message()** [cite: 1063, 1064, 1065].
- [cite_start]**Phần 2: Build RoutingAgent (**host_agent/routing_agent.py**)** [cite: 1096]
 - [cite_start]Xây dựng tác tử trung tâm có khả năng định tuyến yêu cầu của người dùng đến các Agent chuyên biệt (**airbnb_agent** hoặc **weather_agent**) [cite: 1097].
 - [cite_start]Sử dụng mô hình ngôn ngữ tự nhiên và công cụ **send_message()** để giao tiếp với các Agent con theo kiến trúc A2A[cite: 1098].
 - [cite_start]Sử dụng Google ADK SDK (**Agent**, **ToolContext**, **ReadonlyContext**, **CallbackContext**) [cite: 1124, 1127, 1125, 1126].
 - [cite_start]**A2ACardResolver** giúp truy xuất **AgentCard** từ các Agent con qua HTTP[cite: 1131].
 - [cite_start]Lớp **RoutingAgent** khởi tạo các kết nối đến Remote Agent, tạo Agent định tuyến dùng mô hình **gemini-2.5-flash**, và sinh lệnh hành vi cho Agent để định tuyến tác vụ[cite: 1135, 1147, 1164, 1178, 1193].
- [cite_start]**Phần 3: Chạy Host Agent (**host_agent/_main_.py**)** [cite: 1257]
 - [cite_start]Là điểm khởi chạy chính cho toàn bộ hệ thống Host Agent[cite: 1258].
 - [cite_start]Khởi tạo phiên làm việc với **RoutingAgent** và triển khai giao diện Gradio để người dùng tương tác trực tiếp[cite: 1259].
 - [cite_start]Sử dụng **Runner** để thực thi Agent ADK, xử lý phiên và phản hồi[cite: 1274].
 - [cite_start]**InMemorySessionService** lưu trạng thái cuộc trò chuyện trong RAM để duy trì ngữ cảnh[cite: 1276].
 - [cite_start]**gradio** được dùng để xây dựng giao diện người dùng tương tác web (**ChatInterface**) [cite: 1277, 1339, 1341].

- [cite_start]Xử lý các loại phản hồi từ Agent: `function_call`, `function_response`, `is_final_response()` [cite: 1323, 1324, 1325].
- [cite_start]Khi chạy trực tiếp tệp, hệ thống sẽ khởi tạo phiên và triển khai Gradio UI tại địa chỉ `http://localhost:8083` [cite: 1358].

4. Cloning GitHub Repo và chạy Demo

- [cite_start]Mã nguồn mẫu được cung cấp tại: <https://github.com/quaghien/A2A-samples> [cite: 1360].
- [cite_start]**Yêu cầu:** Python 3.13, uv (công cụ quản lý môi trường Python), Node.js (v20), và `.env` file chứa cấu hình API key [cite: 1362, 1363, 1364, 1365].
- [cite_start]Người dùng cần đọc hướng dẫn chi tiết trong README.md của repo để chạy demo [cite: 1368].

Nhờ AI tóm tắt nội dung cần đọc

Bạn hãy liệt kê nội dung một cách logic để tôi có thể hiểu được file này trong vòng 1h. Ví dụ như ban đầu là tổng quan nội dung... sau đó từng bước từng bước là gì? T Tại sao lại có vấn đề phát sinh và solution sinh ra để giải quyết như thế nào?

Tham khảo

1. Tutorial: Agent2Agent (A2A) Protocol: <https://drive.google.com/file/d/1kwuysZALguVPMhr5ozWs6ze-Ab6nBlbl/view>. Hồ Quang Hiến. Nguyễn Quốc Thái. Đinh Quang Vinh. AI VIET NAM – AI COURSE 2025
- 2.